Support Vector Machine

Instructor: Jyotishka Datta

Virginia Tech

April 14, 2025

Today's topic: Support Vector Classifiers

Introduction

- Developed in the computer science community in the 1990s – grown in popularity – considered as one of the best "out of the box" classifier.
- Generalization of a simple and intuitive method called: *maximum margin classifier.*
- Maximum margin classifier requires that classes be separable by linear boundary – not applicable to most data sets.
- Support vector classifier : broader range of cases.
- Support vector machine: further extension of the support vector classifier in order to accommodate non-linear class boundaries.

Beyond SVM

- Support vector machines are intended for the binary classification setting in which there are two classes.
- Can be easily extended to the case of more than two classes.
- They are also closely connected to other statistical methods such as logistic regression.
- Can be also interpreted in a Bayesian sense (Bayesian SVM).
- We will introduce maximum margin classifiers and SVM for the binary classification.

Maximum Margin Classifier

- We first need to know what is a hyperplane and what is a separating hyperplane.
- Consider a p-dimensional space of predictors.
- A hyperplane is a $(p-1)$-dimensional affine space which separates the space into two regions.
- For instance, in 2 dimensions, a hyperplane is a flat one-dimensional subspace – in other words, a line.
- In three dimensions, a hyperplane is a flat two-dimensional subspace–that is, a plane.

Maximum Margin Classifier

- The normal vector $(\beta_1, \ldots, \beta_p)$, is a unit vector $\sum_{j=1}^{p} \beta_j^2 = 1$, which is perpendiculur to the hyperplane.
- If the hyperplane goes through the origin, the (signed) distance between a point $(x_1, \ldots, x_p)$ and the plane is the dot product:

$$x \cdot \beta = x_1\beta_1 + \cdots + x_p\beta_p$$

- The sign of the dot product tells us on which side of the hyperplane the point lies.

Hyperplane in 2-D

The hyperplane $1 + 2X_1 + 3X_2 = 0$ is shown. The blue region is the set of points for which $1 + 2X_1 + 3X_2 > 0$, and the purple region is the set of points for which $1 + 2X_1 + 3X_2 < 0$.

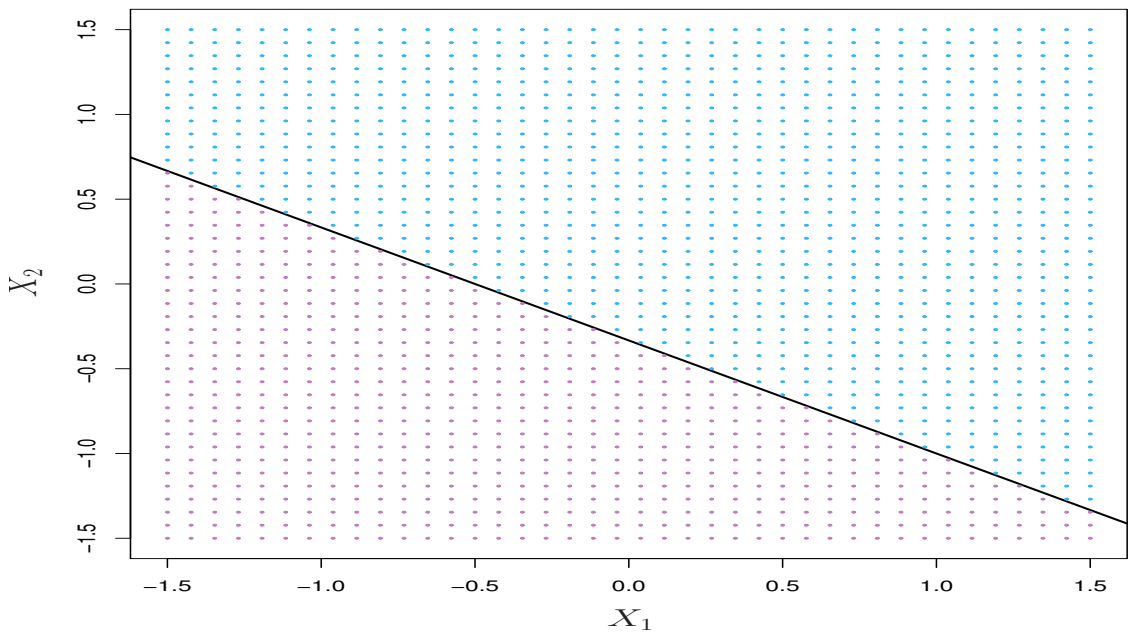
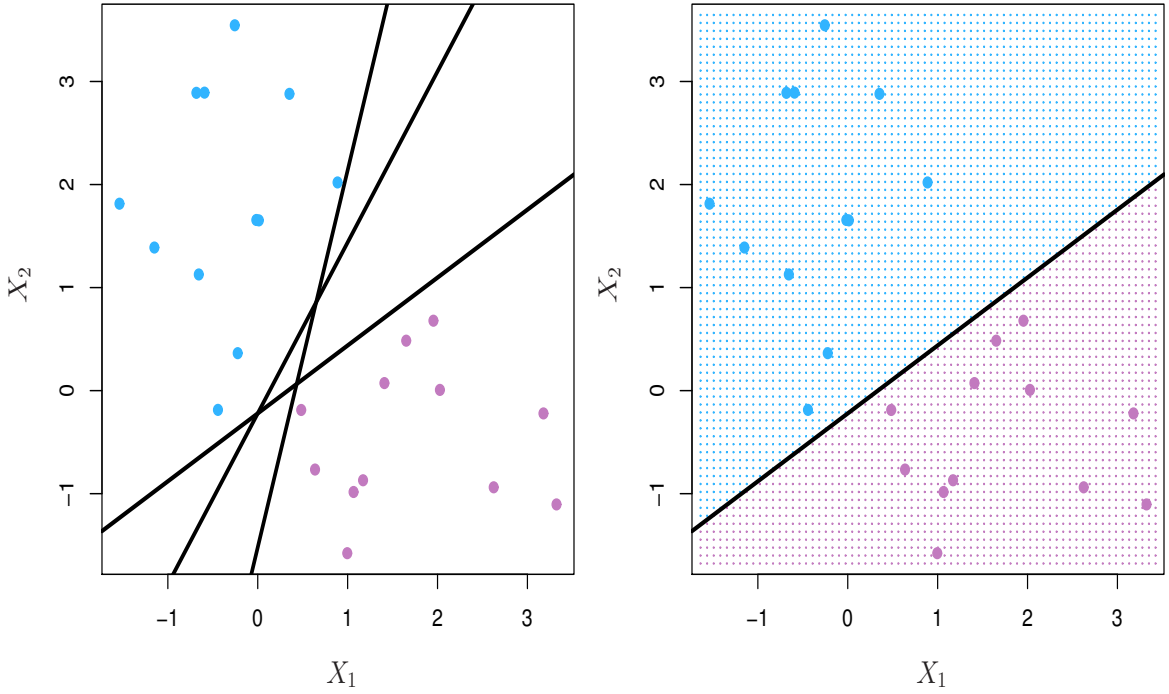


Figure: The black curve indicates the KNN decision boundary using K = 10. The Bayes decision boundary is shown as a purple dashed line. The KNN and Bayes decision boundaries are very similar

Maximum Margin Classifier

- Suppose we have a classification problem with response $Y = 1$ or $Y = -1$.
- Our goal is to construct a classifier with a linear boundary, i.e. with a separating hyperplane.
- If the classes can be separated with a linear boundary, there will typically be **an infinite number of hyperplanes separating the classes.**
- Which classification rule should we choose?

Which classification rule should we choose?



Intuition

- A natural choice is the maximal margin hyperplane (also known as the optimal separating hyperplane), which is the separating hyperplane that is farthest from the training observations.

Intuition

- A natural choice is the maximal margin hyperplane (also known as the optimal separating hyperplane), which is the separating hyperplane that is farthest from the training observations.
- We can compute the (perpendicular) distance from each training observation to a given separating hyperplane; the smallest such distance is the minimal distance from the observations to the hyperplane, and is known as the margin.

Intuition

- A natural choice is the maximal margin hyperplane (also known as the optimal separating hyperplane), which is the separating hyperplane that is farthest from the training observations.
- We can compute the (perpendicular) distance from each training observation to a given separating hyperplane; the smallest such distance is the minimal distance from the observations to the hyperplane, and is known as the margin.
- The maximal margin margin hyperplane is the separating hyperplane for which the margin is largest – that is, it is the hyperplane that has the **farthest minimum distance** to the training observations.

Maximal Margin Classifiers

- Idea: Out of all possible hyperplanes that separate the 2 classes, choose the one with the **widest margin.**
- **Margin** = minimum distance of any point to hyperplane.
- Margin literally represents your classifier margin of error, how far data-points must be shifted before classifier makes a mistake
- Hope is that if margin is large, test data-points will be more likely to fall on correct side of hyperplane.

Which classification rule should we choose?

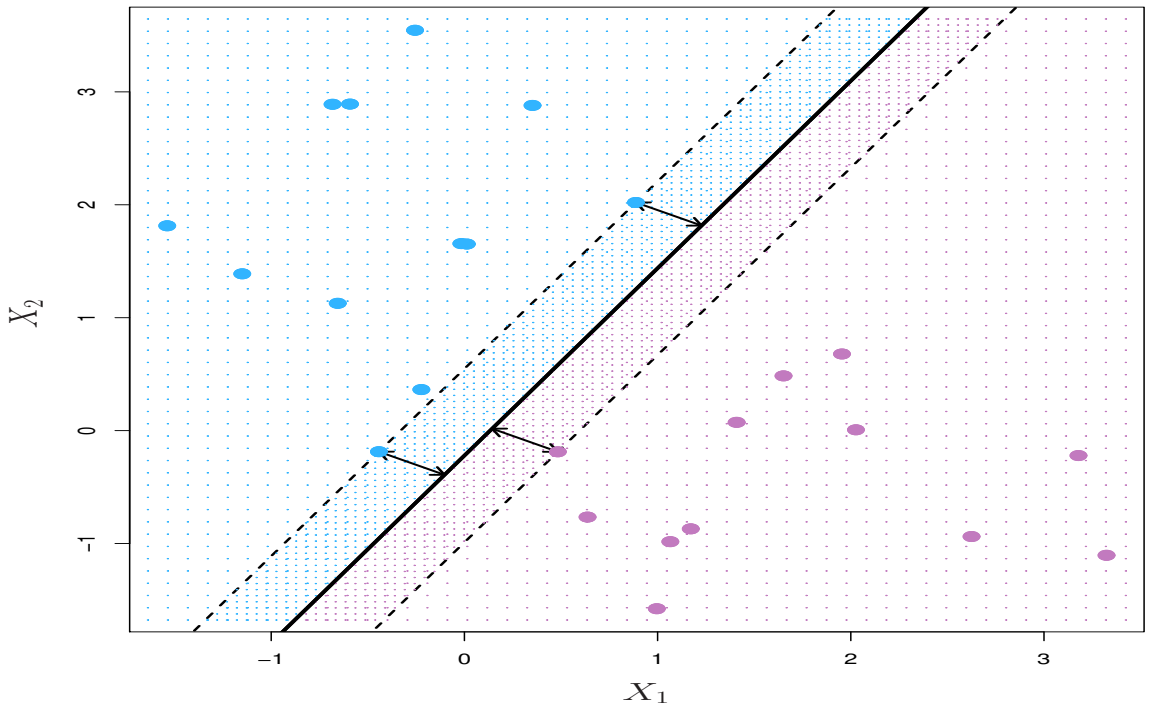


Figure: There are two classes of observations, shown in blue and in purple. The maximal margin hyperplane is shown as a solid line. The margin is the distance from the solid line to either of the dashed lines. The two blue points and the purple point that lie on the dashed lines are the support vectors, and the distance from those points to the hyperplane is indicated by arrows. The purple and blue grid indicates the decision rule made by a classifier based on this separating hyperplane.

Maximal Margin Classifier: Construction

- Training data: $x_1, \ldots, x_n \in \mathbb{R}^p$, class labels: $y_1, \ldots, y_n \in \{-1, 1\}$.
- This can be written as an optimization problem:

$$\max_{\beta_0, \beta_1, \ldots, \beta_p} M$$

$$\text{subject to } \sum_{j=1}^{p} \beta_j^2 = 1$$

$$y_i(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}) \geq M \text{ for all } i = 1, \ldots, n.$$

- M is simply the width of the margin in either direction.
- Once we solve the system, test observation x^* is classified based on sign of $f(x^*) = \beta_0 + \beta_1 x_1^* + \cdots + \beta_p x_p^*$.

Simpler than it looks!

- Condition 3:

$$y_i(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}) \geq M \text{ for all } i = 1, \ldots, n$$

- Guarantees that each observation will be on the correct side of the hyperplane, provided that M is positive.

Simpler than it looks!

- Condition 3:

$$y_i(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}) \geq M \text{ for all } i = 1, \ldots, n$$

- Guarantees that each observation will be on the correct side of the hyperplane, provided that M is positive.
- Condition (2):

$$\sum_{j=1}^{p} \beta_j^2 = 1$$

Simpler than it looks!

- Condition 3:

$$y_i(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}) \geq M \text{ for all } i = 1, \ldots, n$$

- Guarantees that each observation will be on the correct side of the hyperplane, provided that M is positive.
- Condition (2):

$$\sum_{j=1}^{p} \beta_j^2 = 1$$

- This is not really a constraint on the hyperplane.
- If $\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi} = 0$ is a hyperplane, so is $k(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}) = 0$. *But it gives meaning to Condition (3).*

Simpler than it looks!

- One can show that with this constraint the perpendicular distance from the ith observation to the hyperplane is given by

$$y_i(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi})$$

- Basically, these two ensure that each observation is on the correct side of the hyperplane and at least a distance M from the hyperplane.
- Hence, M represents the margin of our hyperplane,
- The optimization problem chooses $\beta_0, \beta_1, \ldots, \beta_p$ to maximize M.
- This is exactly the definition of the maximal margin hyperplane!
- Also, the optimization problem can be solved efficiently. (Use KKT conditions)

Support Vector Classifier

- **Problem 1:** It is not always possible to separate the points using a hyperplane.
- **Problem 2:** Even if it exists, a separating hyperplane based classifier is not always desirable.
- Sensitive to individual observations – overfitting !!

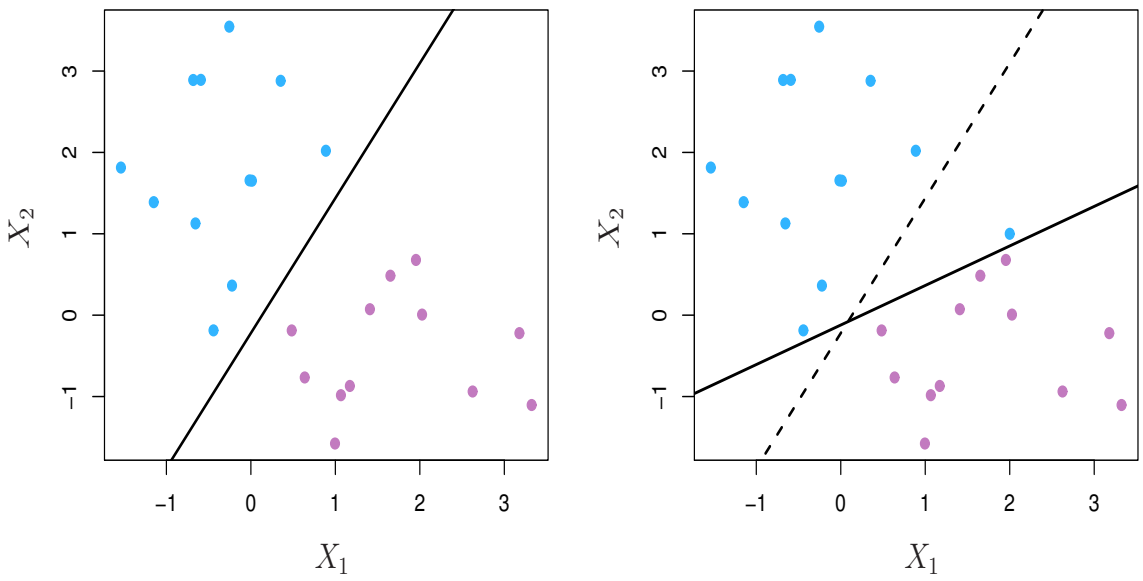


Figure: The addition of a single observation in the right-hand panel leads to a dramatic change in the maximal margin hyperplane.

Support Vector Classifier

- It might make sense to build a classifier that doesn't perfectly separate the two classes. Why?
  1. Robustness to individual observations.
  2. Better classification of *most* of the training observations.
- We are willing to misclassify a few in order to classify the rest better !
- **Support Vector Classifier** aka *Soft margin classifier*
- Relaxation of maximum margin classifier.
- Allows a few points to be on the wrong side of the margin, even on the wrong side of the plane.
- It's inevitable if there's no separating hyperplane.

Support Vector Classifier

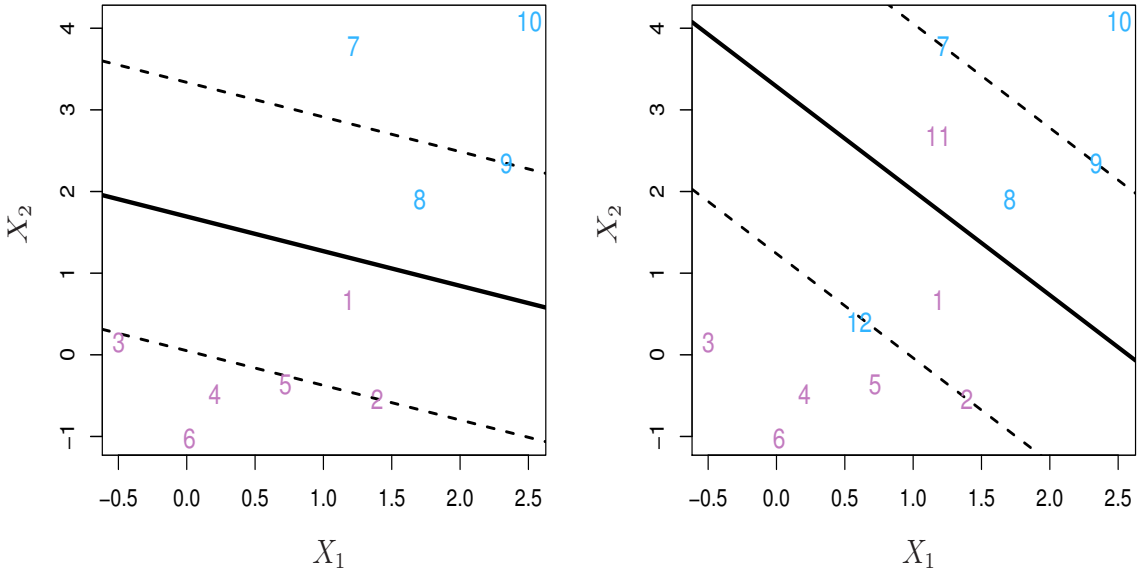


Figure: Left: The hyperplane is shown as a solid line and the margins are shown as dashed lines. Purple observations: observation 1 is on the wrong side of the margin. Blue observations:Observation 8 is on the wrong side of the margin. No observations are on the wrong side of the hyperplane. Right: Same as left panel with two additional points, 11 and 12. **These two observations are on the wrong side of the hyperplane** and the wrong side of the margin.

Details

This can be written as an Optimization problem.

$$\max_{\beta_0, \boldsymbol{\beta}_{1\times p}, \boldsymbol{\epsilon}_{1\times n}} M \tag{1}$$

$$\text{subject to } \sum_{j=1}^{p} \beta_j^2 = 1 \tag{2}$$

$$\underbrace{y_i(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi})}_{\text{distance of } x_i \text{ from the hyperplane}} \geq M(1 - \epsilon_i) \; \forall i = 1, \ldots, n. \tag{3}$$

$$\epsilon_i \geq 0, \sum_{i=1}^{n} \epsilon_i \leq C \tag{4}$$

- M is the width of the margin in either direction.
- $\boldsymbol{\epsilon} = (\epsilon_1, \ldots, \epsilon_n)$ are called Slack variables.
- C is called the budget.

Support Vector Classifiers

- M is the width of the margin - the larger, the better.
- ϵ_i's are slack variables that allow an individual observation to be on the wrong side of the margin or the plane.
- Once we solve the system, test observation x^* is classified based on sign of $f(x^*) = \beta_0 + \beta_1 x_1^* + \cdots + \beta_p x_p^*$.
- At first go, the optimization looks complex, but it's actually pretty simple.

Slack variables

- The slack variable ϵ_i tells us where the ith variable is located with respect to the margin and the hyperplane.

$$\epsilon_i = 0 \Rightarrow i^{th} \text{ observation is on the correct side of the margin}$$
$$\epsilon_i > 0 \Rightarrow i^{th} \text{ observation is on the wrong side of the margin}$$
$$\epsilon_i > 1 \Rightarrow i^{th} \text{ observation is on the wrong side of the plane}$$

- Now look at the parameter C: We have $\sum_{i=1}^{n} \epsilon_i \leq C$.
- C bounds the sum of the ϵ_is, and so it determines the number and severity of the violations to the margin (and to the hyperplane) that we will tolerate.
- C is a budget for the amount that the margin can be violated by the n observations.

Slack variables

- If $C = 0$, no budget for violations $= \epsilon_1 = \cdots = \epsilon_n = 0 =$ maximal margin classifier.
- If $C > 0$ at most C observations can be on the wrong side of the hyperplane. Why?

Slack variables

- If $C = 0$, no budget for violations $= \epsilon_1 = \cdots = \epsilon_n = 0 =$ maximal margin classifier.
- If $C > 0$ at most C observations can be on the wrong side of the hyperplane. Why?
- $\epsilon_i > 1 \Rightarrow i^{th}$ observation is on the wrong side of the plane.

Slack variables

- If $C = 0$, no budget for violations $= \epsilon_1 = \cdots = \epsilon_n = 0 =$ maximal margin classifier.
- If $C > 0$ at most C observations can be on the wrong side of the hyperplane. Why?
- $\epsilon_i > 1 \Rightarrow i^{th}$ observation is on the wrong side of the plane.
- As C increases, we become more tolerant, as C decreases, the margin narrows.

Slack variables

- If $C = 0$, no budget for violations $= \epsilon_1 = \cdots = \epsilon_n = 0 =$ maximal margin classifier.
- If $C > 0$ at most C observations can be on the wrong side of the hyperplane. Why?
- $\epsilon_i > 1 \Rightarrow i^{th}$ observation is on the wrong side of the plane.
- As C increases, we become more tolerant, as C decreases, the margin narrows.
- There must be an optimal C in between !

Slack variables

- If $C = 0$, no budget for violations $= \epsilon_1 = \cdots = \epsilon_n = 0 =$ maximal margin classifier.
- If $C > 0$ at most C observations can be on the wrong side of the hyperplane. Why?
- $\epsilon_i > 1 \Rightarrow i^{th}$ observation is on the wrong side of the plane.
- As C increases, we become more tolerant, as C decreases, the margin narrows.
- There must be an optimal C in between !
- In practice, C is treated as a tuning parameter that is generally chosen via **cross-validation**.

Effect of C

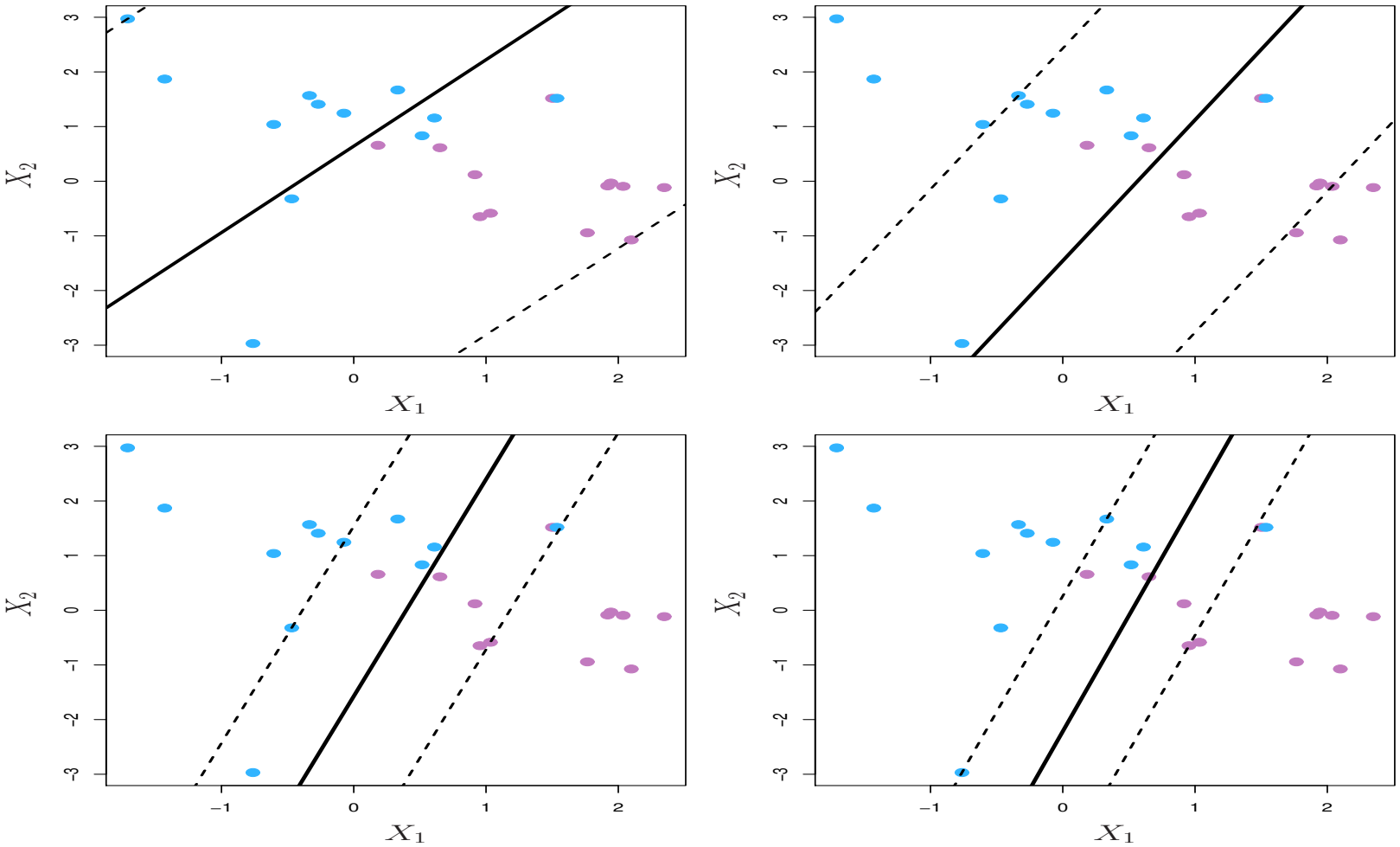


Figure: High C to low C.

Support Vectors

- The optimization problem has a very interesting property:
- it turns out that only observations that either lie on the margin or that violate the margin will affect the hyperplane, and hence the classifier obtained.
- Points strictly on the correct side of the margin does not affect the support vector classifier!
- Changing the position of that observation would not change the classifier at all, provided that its position remains on the correct side of the margin.
- Observations that lie directly on the margin, or on the wrong side of the margin for their class, are known as support vectors.
- These observations do affect the support vector classifier.

$C \propto$ Number of Support Vectors

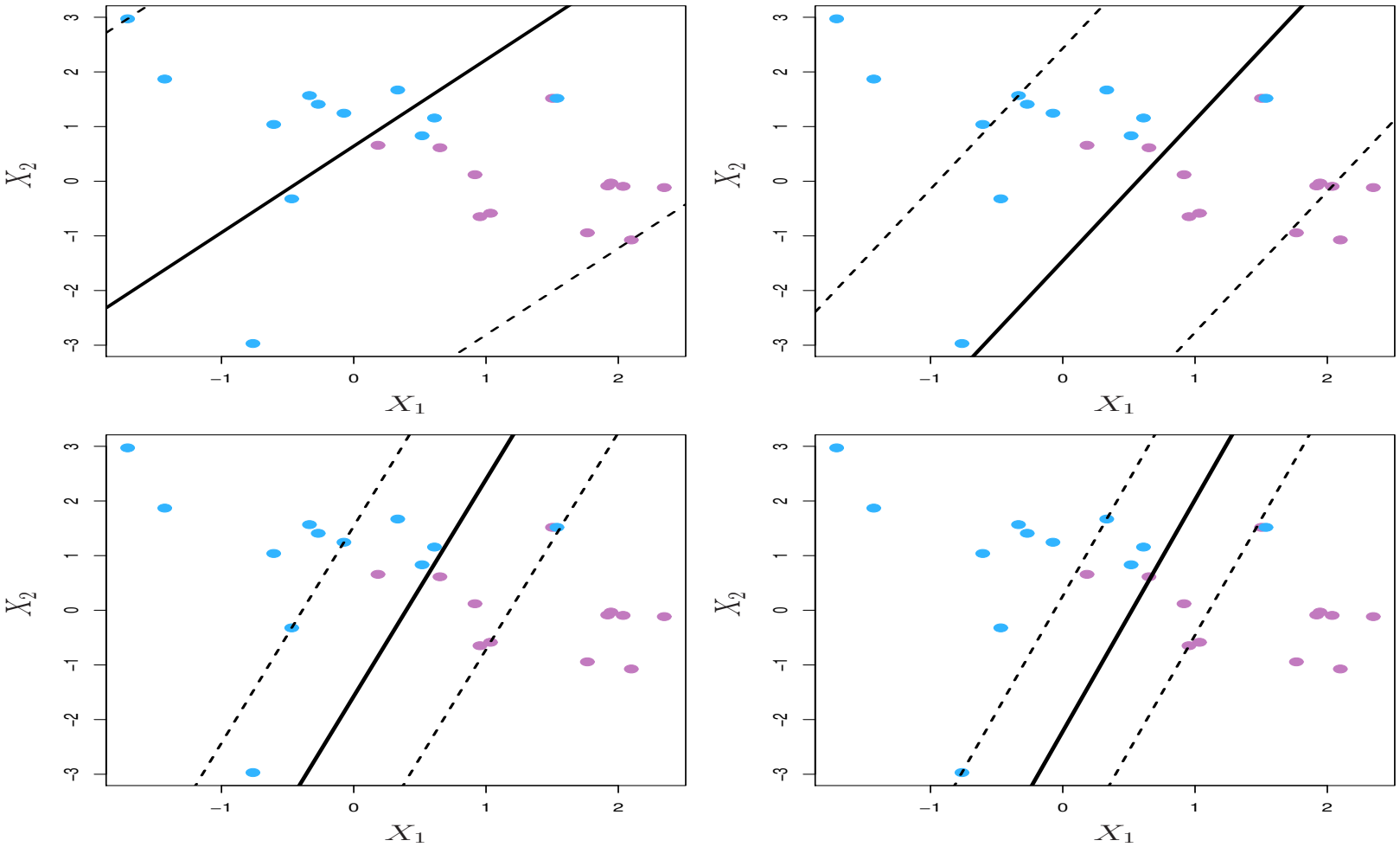


Figure: High C to low C.

Support Vectors and C

- The fact that only support vectors affect the classifier is in line with our previous assertion that C controls the bias-variance trade-off of the support vector classifier.
- When the tuning parameter C is large, then the margin is wide, many observations violate the margin, and so there are many support vectors.
- In this case, many observations are involved in determining the hyperplane - low variance, potentially high bias.
- In contrast, if C is small, then there will be fewer support vectors and hence the resulting classifier will have low bias but high variance.

Sensitivity

- The fact that the support vector classifier's decision rule is based only on a potentially small subset of the training observations (the support vectors) means that it is quite robust to the behavior of observations that are far away from the hyperplane.

Sensitivity

- The fact that the support vector classifier's decision rule is based only on a potentially small subset of the training observations (the support vectors) means that it is quite robust to the behavior of observations that are far away from the hyperplane.
- This distinguishes support vector classifiers from LDA.

Sensitivity

- The fact that the support vector classifier's decision rule is based only on a potentially small subset of the training observations (the support vectors) means that it is quite robust to the behavior of observations that are far away from the hyperplane.
- This distinguishes support vector classifiers from LDA.
- LDA depends on mean of all observations and covariance matrix computed from all observations.

Sensitivity

- The fact that the support vector classifier's decision rule is based only on a potentially small subset of the training observations (the support vectors) means that it is quite robust to the behavior of observations that are far away from the hyperplane.
- This distinguishes support vector classifiers from LDA.
- LDA depends on mean of all observations and covariance matrix computed from all observations.
- Logistic, on the other hand, is not as sensitive.

Non-linear Boundary

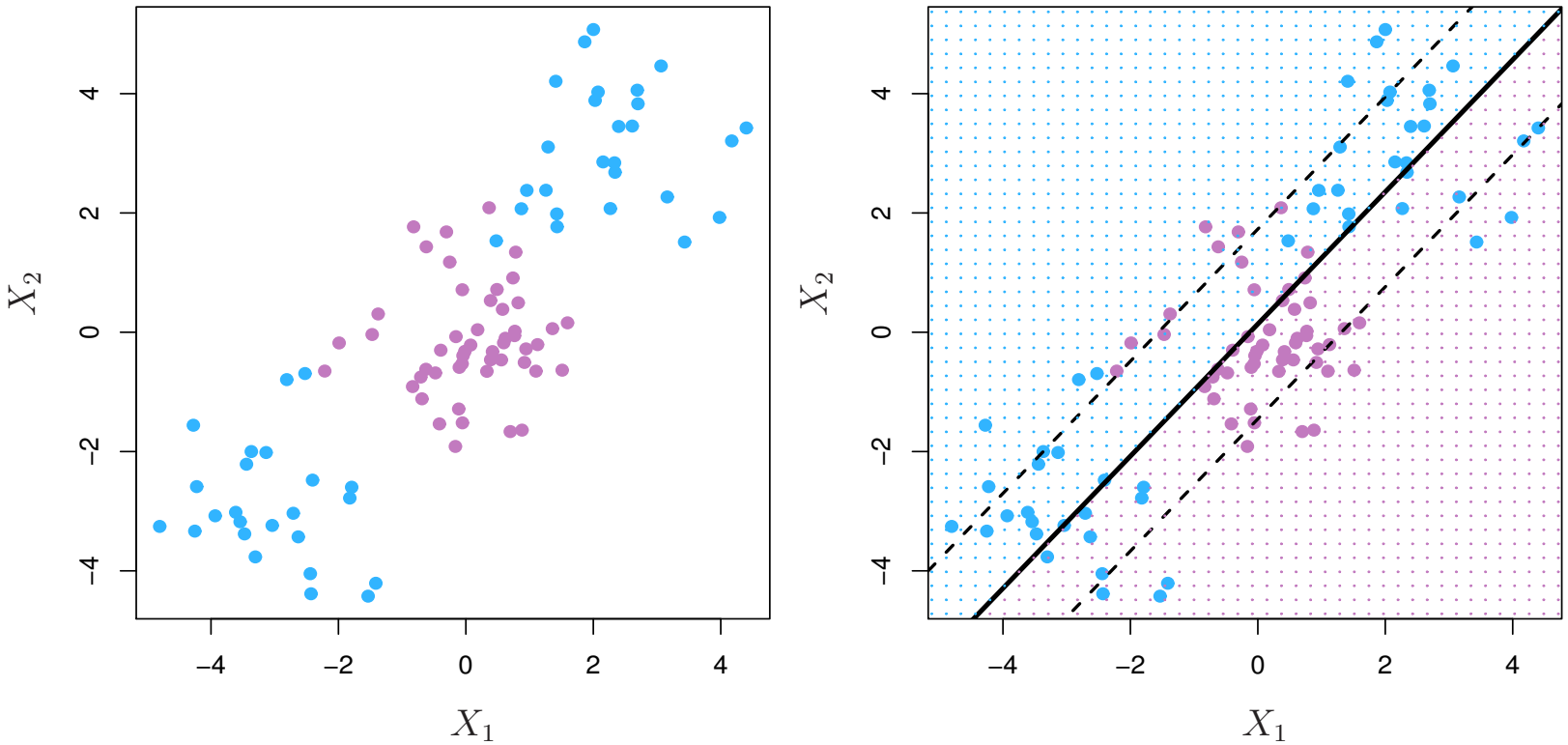


Figure: Left: The observations fall into two classes, with a non-linear boundary between them. Right: The support vector classifier seeks a linear boundary, and consequently performs very poorly

SVM

Linear boundary doesn't work for non-linear problems!
Fix: Enlarge the space of X: using 'functions' like cubic or quadratic terms.
The support vector machine (SVM) is an extension of the support vector classifier that results from enlarging the feature space in a specific way, using **kernels**.

Non-linear Boundary: SVM

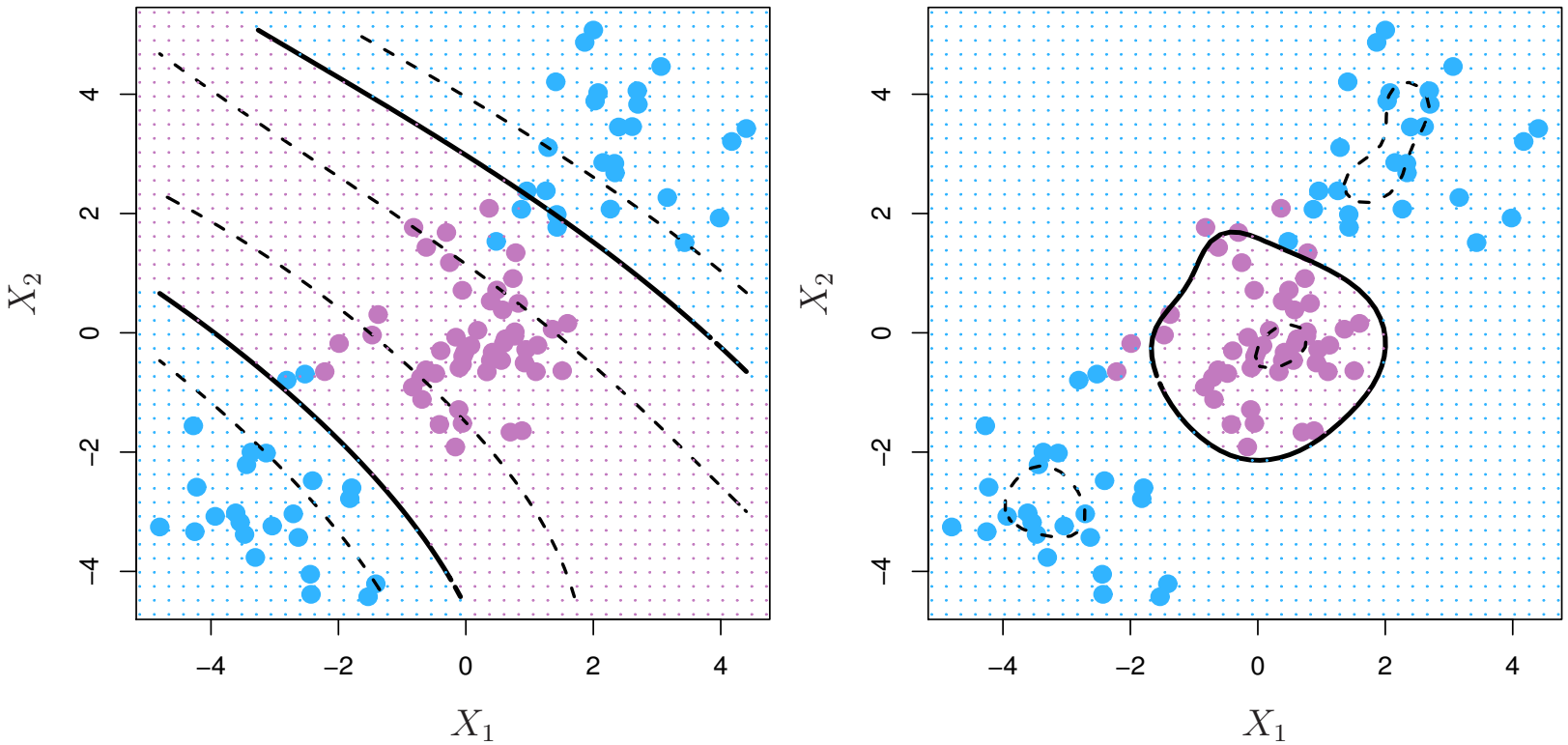


Figure: Left: An SVM with a polynomial kernel of degree 3 is applied to the non-linear data from last figure, resulting in a far more appropriate decision rule. Right: An SVM with a radial kernel is applied. In this example, either kernel is capable of capturing the decision boundary.

Kernel Trick

Feature Expansion Idea[1]

- Consider, for example, the circle data on the left side of Figure **??**: first class forms a circle in the middle of a square, the remaining points form the second class.
- classes do not overlap: but are not perfectly separable by a hyperplane (i.e., a straight line)

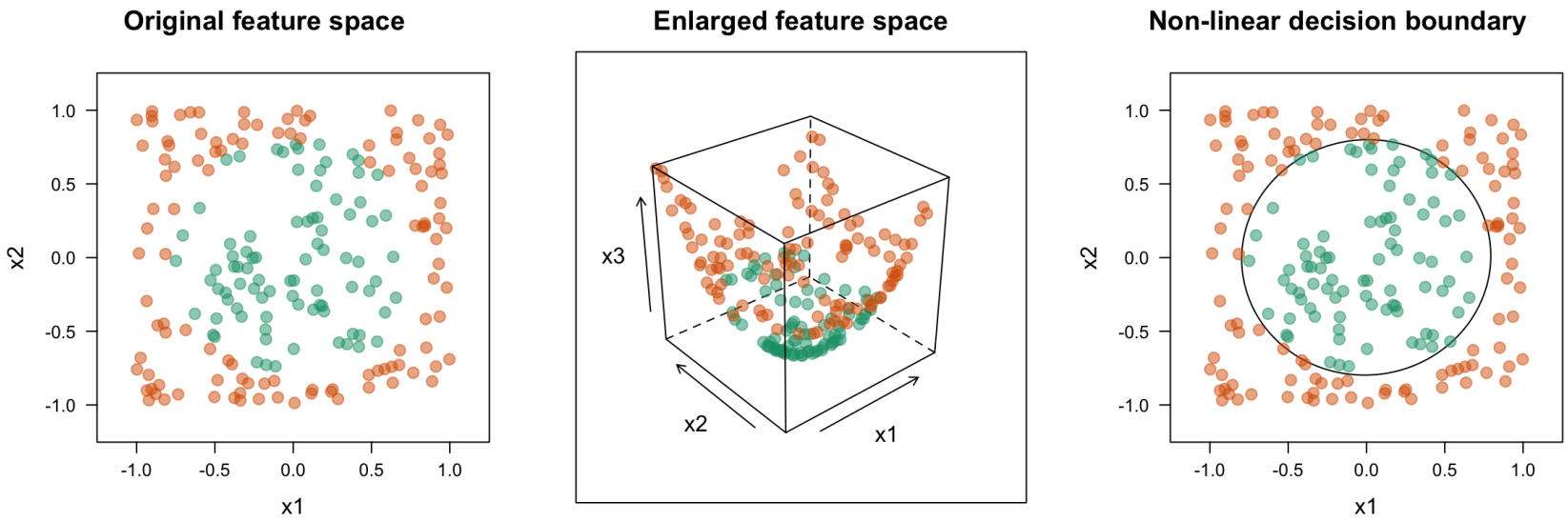


Figure: Simulated nested circle data.

[1] taken from the HOML book

Feature Expansion Idea

- However, we can enlarge the feature space by adding a third feature: $X_3 = X_1^2 + x_2^2$.
- The data are plotted in the enlarged feature space in the middle of Figure ??.
- In this new three dimensional feature space, the two classes are perfectly separable by a hyperplane (i.e., a flat plane);

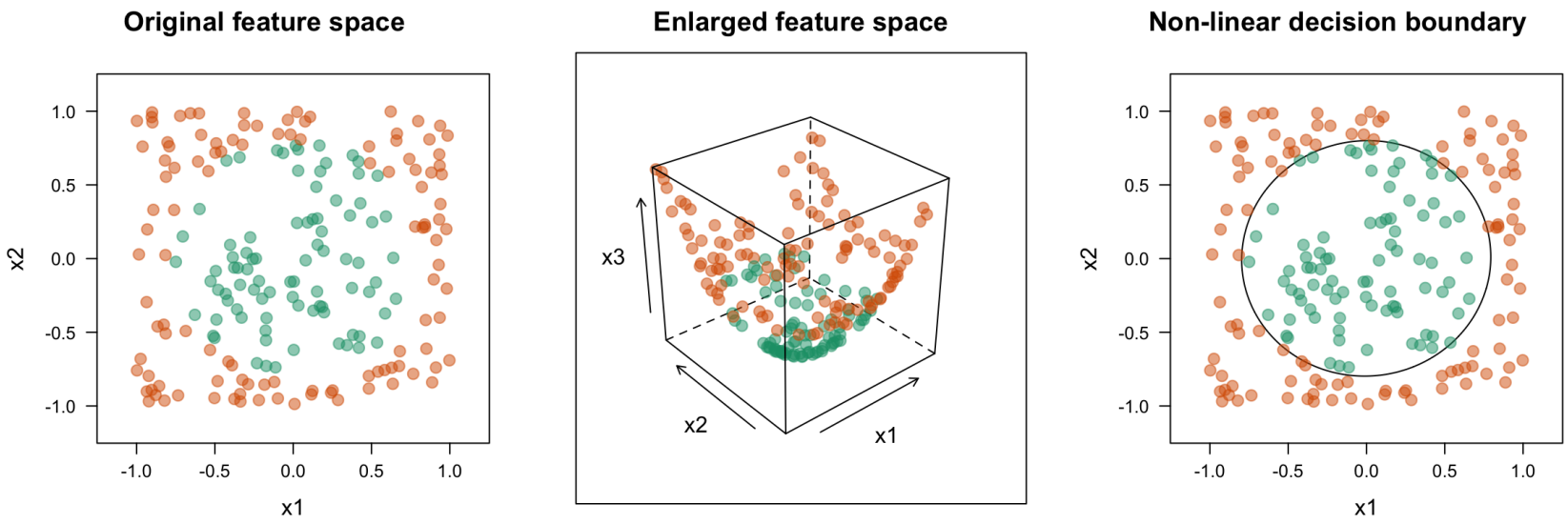


Figure: Simulated nested circle data.

Feature Expansion Idea

- The resulting decision boundary is then projected back onto the original feature space resulting in a non-linear decision boundary which perfectly separates the original data (see the right side of Figure ??)!

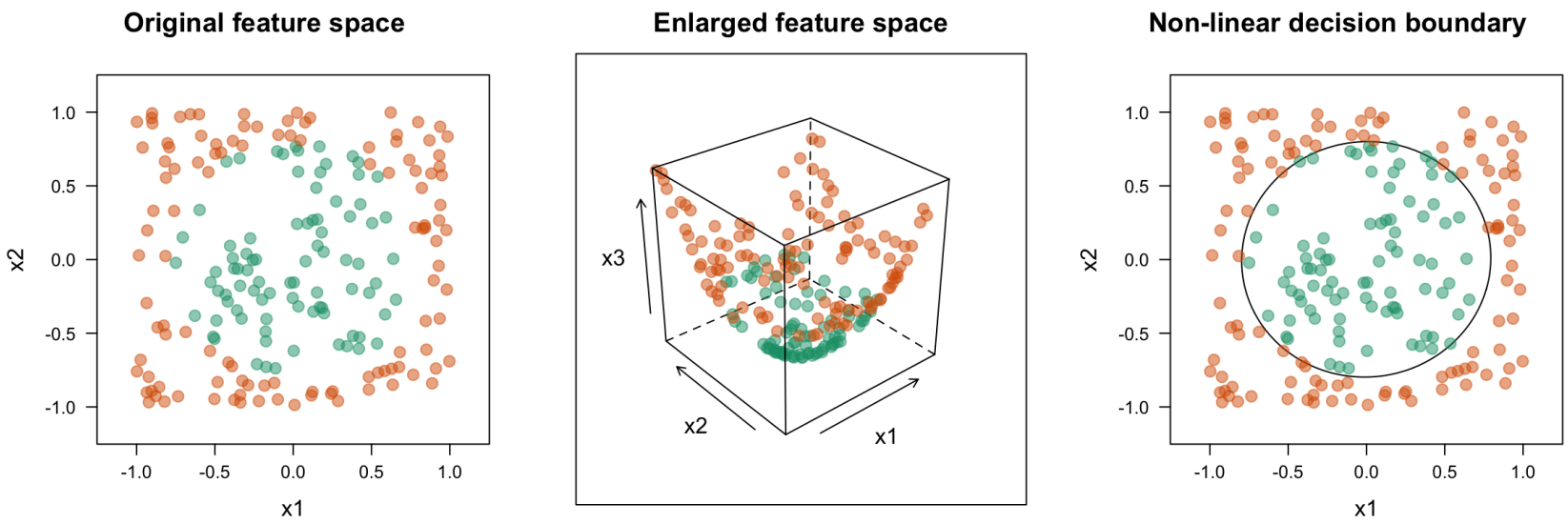


Figure: Simulated nested circle data.

How to deal with non-linear boundaries?

- In **logistic regression**, we dealt with this problem by adding transformations of the predictors.

How to deal with non-linear boundaries?

- In **logistic regression**, we dealt with this problem by adding transformations of the predictors.
- The original decision boundary is a line:

$$\log\left[\frac{P(Y=1|X)}{P(Y=0|X)}\right] = \beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0.$$

How to deal with non-linear boundaries?

- In **logistic regression**, we dealt with this problem by adding transformations of the predictors.
- The original decision boundary is a line:

$$\log\left[\frac{P(Y=1|X)}{P(Y=0|X)}\right] = \beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0.$$

- With a quadratic predictor, we get a quadratic boundary:

$$\log\left[\frac{P(Y=1|X)}{P(Y=0|X)}\right] = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 = 0.$$

How to deal with non-linear boundaries?

- With a **support vector classifier** we can apply a similar trick.

How to deal with non-linear boundaries?

- With a **support vector classifier** we can apply a similar trick.
- The original decision boundary is the hyperplane defined by:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0.$$

How to deal with non-linear boundaries?

- With a **support vector classifier** we can apply a similar trick.
- The original decision boundary is the hyperplane defined by:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0.$$

- If we expand the set of predictors to the 4D space (X_1, X_2, X_1^2, X_2^2), the decision boundary becomes:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 = 0.$$

How to deal with non-linear boundaries?

- With a **support vector classifier** we can apply a similar trick.
- The original decision boundary is the hyperplane defined by:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0.$$

- If we expand the set of predictors to the 4D space (X_1, X_2, X_1^2, X_2^2), the decision boundary becomes:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 = 0.$$

- This is in fact a linear boundary in the augmented variable set (X_1, X_2, X_1^2, X_2^2) but a quadratic boundary in (X_1, X_2).

How do we expand the space of predictors?

- **Idea:** Add polynomial terms up to degree d:

$$Z = (X_1, X_1^2, \ldots, X_1^d, X_2, X_2^2, \ldots, X_2^d, \ldots, X_p, X_p^2, \ldots, X_p^d).$$

How do we expand the space of predictors?

- **Idea:** Add polynomial terms up to degree d:

$$Z = (X_1, X_1^2, \ldots, X_1^d, X_2, X_2^2, \ldots, X_2^d, \ldots, X_p, X_p^2, \ldots, X_p^d).$$

- Does this make the computation more expensive?

How do we expand the space of predictors?

- **Idea:** Add polynomial terms up to degree d:

$$Z = (X_1, X_1^2, \ldots, X_1^d, X_2, X_2^2, \ldots, X_2^d, \ldots, X_p, X_p^2, \ldots, X_p^d).$$

- Does this make the computation more expensive?
- Recall that all we need to compute is the dot product:

$$x_i \cdot x_k = \langle x_i, x_k \rangle = \sum_{j=1}^{p} x_{ij} x_{kj}.$$

How do we expand the space of predictors?

- **Idea:** Add polynomial terms up to degree d:

$$Z = (X_1, X_1^2, \ldots, X_1^d, X_2, X_2^2, \ldots, X_2^d, \ldots, X_p, X_p^2, \ldots, X_p^d).$$

- Does this make the computation more expensive?
- Recall that all we need to compute is the dot product:

$$x_i \cdot x_k = \langle x_i, x_k \rangle = \sum_{j=1}^{p} x_{ij} x_{kj}.$$

- With the expanded set of predictors, we need:

$$z_i \cdot z_k = \langle z_i, z_k \rangle = \sum_{j=1}^{p} \sum_{\ell=1}^{d} x_{ij}^{\ell} x_{kj}^{\ell}.$$

Kernels

The **kernel matrix** defined by $K(i,k) = \langle z_i, z_k \rangle$ for a set of linearly independent vectors $z_1, \ldots, z_n$ is always **positive semi-definite**, i.e. it is symmetric and has no negative eigenvalues.

Kernels

The **kernel matrix** defined by $K(i,k) = \langle z_i, z_k \rangle$ for a set of linearly independent vectors $z_1, \ldots, z_n$ is always **positive semi-definite**, i.e. it is symmetric and has no negative eigenvalues.

Theorem:

If K is a positive definite $n \times n$ matrix, there exist vectors $(z_1, \ldots, z_n)$ in some space $\mathbf{Z}$, such that $K(i,k) = \langle z_i, z_k \rangle$.

The kernel trick

Example: Suppose that you want to include all linear and quadratic features, including pairwise interactions, in your feature expansion:

$$\begin{aligned}\Phi(X) =& (1, \sqrt{2}X_1, \ldots, \sqrt{2}X_p, \\ & X_1^2, \ldots, X_p^2, \\ & \sqrt{2}X_1X_2, \sqrt{2}X_1X_3, \ldots, \sqrt{2}X_{p-1}X_p)\end{aligned}$$

Computing $K(i,k)$ via $\langle \Phi(X_i), \Phi(X_k) \rangle$ takes order p^2 time.

Equivalently we can compute

$$K(i,k) = k(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^2$$

which takes $O(p)$ time!

$k(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^2$ is a degree 2 **polynomial kernel**.

The kernel trick

Expand the set of predictors:

The kernel trick

Expand the set of predictors:

- Find a mapping Φ which expands the original set of predictors $X_1, \ldots, X_p$. For example,

$$\Phi(X) = (X_1, X_2, X_1^2, X_2^2)$$

The kernel trick

Expand the set of predictors:

- Find a mapping Φ which expands the original set of predictors $X_1, \ldots, X_p$. For example,

$$\Phi(X) = (X_1, X_2, X_1^2, X_2^2)$$

- For each pair of samples, compute:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

The kernel trick

Expand the set of predictors:

- Find a mapping Φ which expands the original set of predictors $X_1, \ldots, X_p$. For example,

$$\Phi(X) = (X_1, X_2, X_1^2, X_2^2)$$

- For each pair of samples, compute:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

Define a kernel:

The kernel trick

Expand the set of predictors:

- Find a mapping Φ which expands the original set of predictors $X_1, \ldots, X_p$. For example,

$$\Phi(X) = (X_1, X_2, X_1^2, X_2^2)$$

- For each pair of samples, compute:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

Define a kernel:

- Find a positive definite function $k(\cdot,\cdot)$. For example:

$$k(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^2 .$$

The kernel trick

Expand the set of predictors:

- Find a mapping Φ which expands the original set of predictors $X_1, \ldots, X_p$. For example,

$$\Phi(X) = (X_1, X_2, X_1^2, X_2^2)$$

- For each pair of samples, compute:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

Define a kernel:

- Find a positive definite function $k(\cdot,\cdot)$. For example:

$$k(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^2 .$$

- For each pair of samples, compute:

$$K(i,k) = k(x_i, x_k).$$

The kernel trick

Expand the set of predictors:

- Find a mapping Φ which expands the original set of predictors $X_1, \ldots, X_p$. For example,

$$\Phi(X) = (X_1, X_2, X_1^2, X_2^2)$$

- For each pair of samples, compute:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

Define a kernel:

- Find a positive definite function $k(\cdot,\cdot)$. For example:

$$k(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^2 .$$

- For each pair of samples, compute:

$$K(i,k) = k(x_i, x_k).$$

- Often much easier!

How are kernels defined?

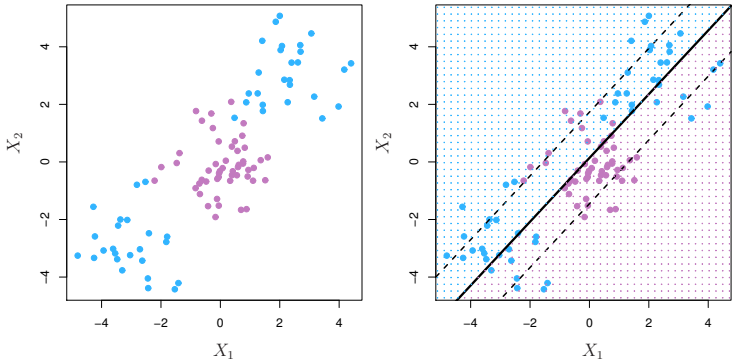
- Proving that a bilinear function $k(\cdot, \cdot)$ is positive definite (PD) is not always easy.

How are kernels defined?

- Proving that a bilinear function $k(\cdot, \cdot)$ is positive definite (PD) is not always easy.
- However, we can easily define PD kernels by combining those we are familiar with:
  - Sums and products of PD kernels are PD.

How are kernels defined?

- Proving that a bilinear function $k(\cdot, \cdot)$ is positive definite (PD) is not always easy.
- However, we can easily define PD kernels by combining those we are familiar with:
  - Sums and products of PD kernels are PD.
- Intuitively, a kernel $K(x_i, x_k)$ defines a *similarity* between the samples x_i and x_k. This intuition can guide our choice in different problems.



Common kernels

- The polynomial kernel:

$$K(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^d$$

Common kernels

- The polynomial kernel:

$$K(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^d$$

- The radial basis kernel:

$$K(x_i, x_k) = \exp\Big(-\gamma \underbrace{\sum_{j=1}^{p} (x_{ip} - x_{kp})^2}_{\text{Euclidean } d(x_i, x_k)} \Big)$$

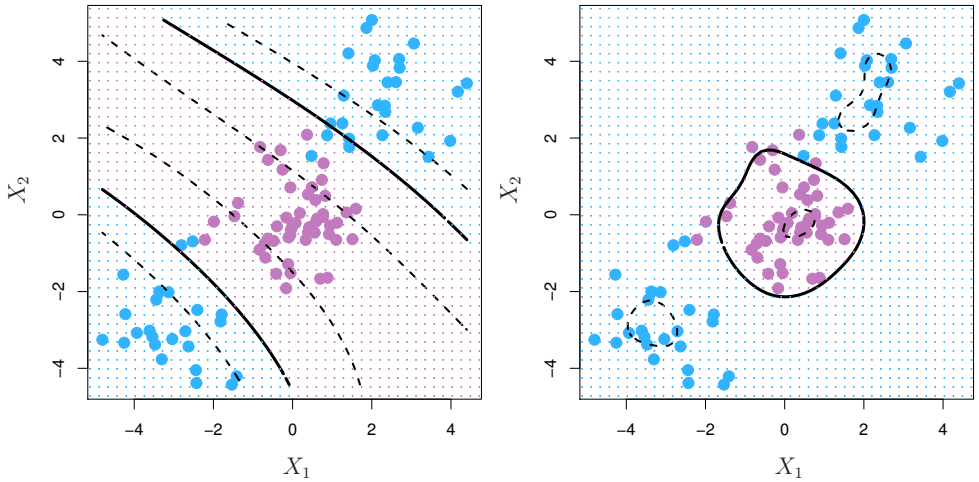
Common kernels

- The polynomial kernel:

$$K(x_i, x_k) = (1 + \langle x_i, x_k \rangle)^d$$

- The radial basis kernel:

$$K(x_i, x_k) = \exp\Big(-\gamma \underbrace{\sum_{j=1}^{p} (x_{ip} - x_{kp})^2}_{\text{Euclidean } d(x_i, x_k)} \Big)$$



What we know so far about support vector machines

- A support vector classifier is a generalization of the max margin classifier that allows the margin to be violated by an amount governed by a budget.
- A **support vector machine** is a support vector classifier applied on an expanded set of predictors, e.g.

$$\Phi : (X_1, X_2) \to (X_1, X_2, X_1X_2, X_1^2, X_2^2).$$

What we know so far about support vector machines

- A support vector classifier is a generalization of the max margin classifier that allows the margin to be violated by an amount governed by a budget.
- A **support vector machine** is a support vector classifier applied on an expanded set of predictors, e.g.

$$\Phi : (X_1, X_2) \to (X_1, X_2, X_1X_2, X_1^2, X_2^2).$$

- One way to fit a SVM is to explicitly apply the feature map Φ to each input vector x_i and then run the support vector classifier on the augmented observations $(\Phi(x_i), y_i)$.

What we know so far about support vector machines

- A support vector classifier is a generalization of the max margin classifier that allows the margin to be violated by an amount governed by a budget.
- A **support vector machine** is a support vector classifier applied on an expanded set of predictors, e.g.

$$\Phi : (X_1, X_2) \to (X_1, X_2, X_1X_2, X_1^2, X_2^2).$$

- One way to fit a SVM is to explicitly apply the feature map Φ to each input vector x_i and then run the support vector classifier on the augmented observations $(\Phi(x_i), y_i)$.
- However, we only need to know the dot products:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i, k)$$

for every pair of samples (x_i, x_k).

The kernel trick

- Often, the dot product:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i,k)$$

  is an easily computed function $k(x_i, x_k)$ of the original vectors, even if the mapping Φ significantly expands the space of predictors.

The kernel trick

- Often, the dot product:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i,k)$$

  is an easily computed function $k(x_i, x_k)$ of the original vectors, even if the mapping Φ significantly expands the space of predictors.

- **Example 1:** Polynomial kernel of degree 2

$$K(i,k) = (1 + \langle x_i, x_k \rangle)^2.$$

The kernel trick

- Often, the dot product:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i,k)$$

  is an easily computed function $k(x_i, x_k)$ of the original vectors, even if the mapping Φ significantly expands the space of predictors.

- **Example 1:** Polynomial kernel of degree 2

$$K(i,k) = (1 + \langle x_i, x_k \rangle)^2.$$

- With two predictors, this corresponds to the mapping:

$$\Phi : (X_1, X_2) \to (1, \sqrt{2}X_1, \sqrt{2}X_2, \sqrt{2}X_1X_2, X_1^2, X_2^2).$$

The kernel trick

- Often, the dot product:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i,k)$$

  is an easily computed function $k(x_i, x_k)$ of the original vectors, even if the mapping Φ significantly expands the space of predictors.

- **Example 2:** RBF kernel

$$K(i,k) = \exp(-\gamma d(x_i, x_k)^2),$$

  where d is the Euclidean distance between x_i and x_k.

The kernel trick

- Often, the dot product:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i,k)$$

  is an easily computed function $k(x_i, x_k)$ of the original vectors, even if the mapping Φ significantly expands the space of predictors.

- **Example 2:** RBF kernel

$$K(i,k) = \exp(-\gamma d(x_i, x_k)^2),$$

  where d is the Euclidean distance between x_i and x_k.

- In this case, the mapping Φ is an expansion into an infinite number of features!

The kernel trick

- Often, the dot product:

$$\langle \Phi(x_i), \Phi(x_k) \rangle \equiv K(i,k)$$

  is an easily computed function $k(x_i, x_k)$ of the original vectors, even if the mapping Φ significantly expands the space of predictors.

- **Example 2:** RBF kernel

$$K(i,k) = \exp(-\gamma d(x_i, x_k)^2),$$

  where d is the Euclidean distance between x_i and x_k.

- In this case, the mapping Φ is an expansion into an infinite number of features! We can apply the method even if we don't what these transformations are.

The kernel trick

- If we don't know which transformations we are using, why would we expect the SVM to work?
  - The kernel $K(i,k) = k(x_i, x_k)$ measures the similarity between samples x_i and x_k.
  - We can evaluate whether k is a good measure of similarity without understanding the feature expansion Φ.

The kernel trick

- If we don't know which transformations we are using, why would we expect the SVM to work?
  - The kernel $K(i,k) = k(x_i, x_k)$ measures the similarity between samples x_i and x_k.
  - We can evaluate whether k is a good measure of similarity without understanding the feature expansion Φ.
- Remember that not every similarity function is a valid kernel function. A valid kernel function will give rise to a positive semidefinite matrix K whenever it is applied to a collection of input vectors, where $K(i,k) = k(x_i, x_k)$.

The kernel trick

- If we don't know which transformations we are using, why would we expect the SVM to work?
  - The kernel $K(i,k) = k(x_i, x_k)$ measures the similarity between samples x_i and x_k.
  - We can evaluate whether k is a good measure of similarity without understanding the feature expansion Φ.
- Remember that not every similarity function is a valid kernel function. A valid kernel function will give rise to a positive semidefinite matrix K whenever it is applied to a collection of input vectors, where $K(i,k) = k(x_i, x_k)$.
- Fortunately, it is not too hard to show that many useful functions k are positive semi-definite.

The kernel trick

- If we don't know which transformations we are using, why would we expect the SVM to work?
  - The kernel $K(i,k) = k(x_i, x_k)$ measures the similarity between samples x_i and x_k.
  - We can evaluate whether k is a good measure of similarity without understanding the feature expansion Φ.
- Remember that not every similarity function is a valid kernel function. A valid kernel function will give rise to a positive semidefinite matrix K whenever it is applied to a collection of input vectors, where $K(i,k) = k(x_i, x_k)$.
- Fortunately, it is not too hard to show that many useful functions k are positive semi-definite.
- Conversely, if the matrix K is positive semi-definite, then there exists *some* mapping Φ to *some* feature space, such that $K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle$.

Kernels for non-standard data types

- We can define families of kernels (with tuning parameters), which capture similarity between non-standard kinds of data:
  1. Text, strings (e.g. documents or tweets)
  2. Molecules (e.g. proteins or DNA)
  3. Images
  4. Graphs (e.g. social network graphs)
  5. Histograms

Kernels for non-standard data types

- We can define families of kernels (with tuning parameters), which capture similarity between non-standard kinds of data:
  1. Text, strings (e.g. documents or tweets)
  2. Molecules (e.g. proteins or DNA)
  3. Images
  4. Graphs (e.g. social network graphs)
  5. Histograms
- Sometimes we know the mapping Φ, but there are algorithms that are fast for computing $K(i, k)$ without doing the expansion explicitly.

Kernels for non-standard data types

- We can define families of kernels (with tuning parameters), which capture similarity between non-standard kinds of data:
  1. Text, strings (e.g. documents or tweets)
  2. Molecules (e.g. proteins or DNA)
  3. Images
  4. Graphs (e.g. social network graphs)
  5. Histograms
- Sometimes we know the mapping Φ, but there are algorithms that are fast for computing $K(i, k)$ without doing the expansion explicitly.
- Other times, the expansion Φ is infinite-dimensional or simply not known, but we can still compute the kernel function.

The kernel trick can be applied beyond SVMs

Kernel PCA:

The kernel trick can be applied beyond SVMs

Kernel PCA:

- Suppose we want to do PCA with an expanded set of predictors, defined by the mapping Φ.
  - Can replace each x_i with $\Phi(x_i)$ in the data matrix
  - This allows us to find nonlinear curves that best explain data variability.

The kernel trick can be applied beyond SVMs

Kernel PCA:

- Suppose we want to do PCA with an expanded set of predictors, defined by the mapping Φ.
  - Can replace each x_i with $\Phi(x_i)$ in the data matrix
  - This allows us to find nonlinear curves that best explain data variability.
- To compute the PC scores of each datapoint, all we need to know is the kernel or Gram matrix:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

The kernel trick can be applied beyond SVMs

Kernel PCA:

- Suppose we want to do PCA with an expanded set of predictors, defined by the mapping Φ.
  - Can replace each x_i with $\Phi(x_i)$ in the data matrix
  - This allows us to find nonlinear curves that best explain data variability.
- To compute the PC scores of each datapoint, all we need to know is the kernel or Gram matrix:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

- Even if Φ expands the predictors to a very high dimensional space, we can do PCA!

The kernel trick can be applied beyond SVMs

Kernel PCA:

- Suppose we want to do PCA with an expanded set of predictors, defined by the mapping Φ.
  - Can replace each x_i with $\Phi(x_i)$ in the data matrix
  - This allows us to find nonlinear curves that best explain data variability.
- To compute the PC scores of each datapoint, all we need to know is the kernel or Gram matrix:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

- Even if Φ expands the predictors to a very high dimensional space, we can do PCA!
- The cost only depends on the number of observations n.

The kernel trick can be applied beyond SVMs

Kernel PCA:

- Suppose we want to do PCA with an expanded set of predictors, defined by the mapping Φ.
  - Can replace each x_i with $\Phi(x_i)$ in the data matrix
  - This allows us to find nonlinear curves that best explain data variability.
- To compute the PC scores of each datapoint, all we need to know is the kernel or Gram matrix:

$$K(i,k) = \langle \Phi(x_i), \Phi(x_k) \rangle.$$

- Even if Φ expands the predictors to a very high dimensional space, we can do PCA!
- The cost only depends on the number of observations n.

Kernel ridge regression, Kernel logistic regression, Kernel linear discriminant analysis, Kernel K-means, . . .